

TUGAS PRAKTIKUM 4
SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.K

Disusun Oleh:

Nama : Muhammad Fahrur Rohman
Nim :2410651056
Prodi : Teknik Informatika
Tanggal : 6 November 2025

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

Tugas Praktikum 4

Tugas 1: Catat Data

DATA SET 1

FCFS

```
fahru@FAHRUR: ~/praktikum x + v
fahru@FAHRUR:~/praktikum4$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23
```

SJF

```
fahru@FAHRUR:~/praktikum4$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

Round Robin

```
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 3

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 7
Waktu 3: Menjalankan P1 -> Sisa waktu: 4
Waktu 6: Menjalankan P2 -> Sisa waktu: 2
Waktu 9: Menjalankan P3 -> Sisa waktu: 5
Waktu 12: Menjalankan P1 -> Sisa waktu: 1
Waktu 15: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P2 -> Selesai pada waktu 18

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    16     16     6
P2      1      5    18     17    12
P3      2      8   -454300256  25496  -454311479

Average Turnaround Time: 8509.67
Average Waiting Time: -151437152.00
```

DATA SET 2

FCFS

```
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5      5      5      0
P2      1      3      8      7      4
P3      2      8     16     14      6
P4      3      6     22     19     13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   5   8  16  22
```

SJF

```
fahru@FAHRUR:~/praktikum4$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== HASIL PENJADWALAN SJF ===


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 5  | 5  | 5   | 0  |
| P2  | 1  | 3  | 8  | 7   | 4  |
| P3  | 2  | 8  | 22 | 20  | 12 |
| P4  | 3  | 6  | 14 | 11  | 5  |



Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
```

Round Robin

```
fahru@FAHRUR:~/praktikum4$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 3

Proses P1
  Arrival Time: 0
  Burst Time: 5

Proses P2
  Arrival Time: 1
  Burst Time: 3

Proses P3
  Arrival Time: 2
  Burst Time: 8

Proses P4
  Arrival Time: 3
  Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 2
Waktu 3: Menjalankan P1 -> Selesai pada waktu 5
Waktu 5: Menjalankan P2 -> Selesai pada waktu 8
Waktu 8: Menjalankan P3 -> Sisa waktu: 5
Waktu 11: Menjalankan P4 -> Sisa waktu: 3
Waktu 14: Menjalankan P1 -> Selesai pada waktu 14
Waktu 14: Menjalankan P3 -> Sisa waktu: 2
Waktu 17: Menjalankan P3 -> Selesai pada waktu 19

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT    | TAT        | WT    |
|-----|----|----|-------|------------|-------|
| P1  | 0  | 5  | 14    | 14         | 9     |
| P2  | 1  | 3  | 8     | 7          | 4     |
| P3  | 2  | 8  | 19    | 17         | 9     |
| P4  | 3  | 6  | 22290 | -854384023 | 22290 |



Average Turnaround Time: -213595984.00
Average Waiting Time: 5578.00
```

Tugas 2

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

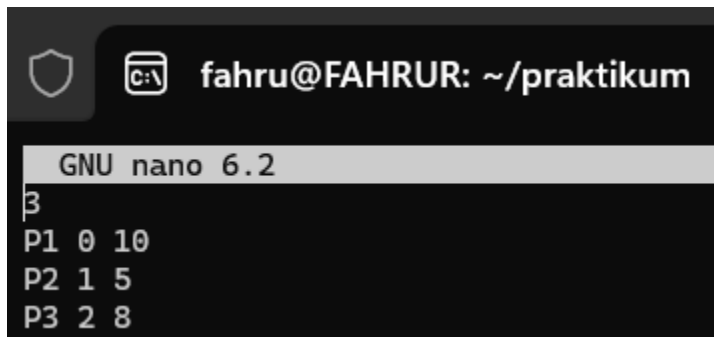
Jawaban:

1. SJF.
2. Karena setiap algoritma memiliki metodenya tersendiri dalam mengatur urutan suatu proses.
3. Saat sistemnya sederhana dan tidak butuh proses cepat.
4. Gunakan SJF jika ingin sistemnya berjalan efisien dan cepat.
5. Gunakan Round Robin jika ingin sistem yang multitasking dan interaktif yang mana semua proses bergiliran.

Tugas 3

Pada tugas 3 ini saya menambahkan beberapa fitur ke salah satu program yaitu fcfs dengan beberapa fitunya antara lain Adalah membaca input dari file teks, menyimpan hasilnya ke file teks dan menampilkan Grant Chart yang lebih detail.

Pertama, saya akan buat file teksnya terlebih dahulu yang akan jadi input



```
fahru@FAHRUR: ~/praktikum
GNU nano 6.2
3
P1 0 10
P2 1 5
P3 2 8
```

Kemudian untuk tambahan fiturnya ialah memodifikasi file FCFSnya yaitu dengan isi kodenya seperti berikut

```

#include <stdio.h>
#include <stdlib.h>

struct Process {
    char id[5];
    int arrival, burst;
    int waiting, turnaround, completion;
};

int main() {
    FILE *fin, *fout;
    fin = fopen("FCFS.txt", "r");
    fout = fopen("hasil_fcfs.txt", "w");

    if (fin == NULL) {
        printf("Gagal membuka file input.txt!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    }

    fclose(fin);

    // FCFS Scheduling
    int current_time = 0;
    float total_wt = 0, total_tat = 0;

    printf("\n=== Hasil Penjadwalan FCFS ===\n");
    fprintf(fout, "=== Hasil Penjadwalan FCFS ===\n");

    printf("\nTabel Hasil:\n");
    printf("ID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(fout, "\nID\tAT\tBT\tCT\tTAT\tWT\n");

    for (int i = 0; i < n; i++) {
        if (current_time < p[i].arrival)
            current_time = p[i].arrival;

        p[i].completion = current_time + p[i].burst;
        p[i].turnaround = p[i].completion - p[i].arrival;
        p[i].waiting = p[i].turnaround - p[i].burst;
        current_time = p[i].completion;

        total_wt += p[i].waiting;
        total_tat += p[i].turnaround;

        printf("%s\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
        fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
    }

    float avg_wt = total_wt / n;
    float avg_tat = total_tat / n;

    printf("\nRata-rata Waiting Time: %.2f", avg_wt);
    printf("\nRata-rata Turnaround Time: %.2f\n", avg_tat);
    fprintf(fout, "\nRata-rata Waiting Time: %.2f", avg_wt);
    fprintf(fout, "\nRata-rata Turnaround Time: %.2f\n", avg_tat);

    // Gantt Chart lebih detail
    printf("\nGantt Chart:\n|");
    fprintf(fout, "\nGantt Chart:\n|");

    int start = 0;
    for (int i = 0; i < n; i++) {
        if (i == 0)
            start = p[i].arrival;
        printf(" %s (%d-%d) |", p[i].id, start, p[i].completion);
        fprintf(fout, " %s (%d-%d) |", p[i].id, start, p[i].completion);
        start = p[i].completion;
    }

    printf("\n\nHasil juga disimpan di file hasil_fcfs.txt\n");
    fprintf(fout, "\n");

    fclose(fout);
    return 0;
}

```

Kemudian compile dan jalankan sehingga hasilnya seperti dibawah ini

```
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  fcfs_modif  fcfs_modif.c
fahru@FAHRUR:~/praktikum4/tugas$ ./fcfs_modif

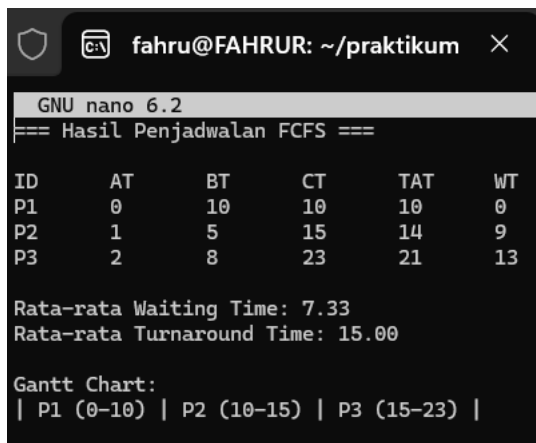
=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
ID    AT    BT    CT    TAT    WT
P1     0    10    10    10     0
P2     1     5    15    14     9
P3     2     8    23    21    13

Rata-rata Waiting Time: 7.33
Rata-rata Turnaround Time: 15.00

Gantt Chart:
| P1 (0-10) | P2 (10-15) | P3 (15-23) |

Hasil juga disimpan di file hasil_fcfs.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt
```



```
GNU nano 6.2
=== Hasil Penjadwalan FCFS ===

ID    AT    BT    CT    TAT    WT
P1     0    10    10    10     0
P2     1     5    15    14     9
P3     2     8    23    21    13

Rata-rata Waiting Time: 7.33
Rata-rata Turnaround Time: 15.00

Gantt Chart:
| P1 (0-10) | P2 (10-15) | P3 (15-23) |
```

Bisa dilihat berdasarkan gambar diatas bahwa hasilnya disimpan ke dalam bentuk file teks.

Tugas 4

Test Case 1 Proses dengan burst time bervariasi

FCS

Masukkan data ke file input txt

```
fahru@FAHRUR: ~/praktikum
GNU nano 6.2
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
|
```

Hasil

```
FCFS.txt Round_Robin.txt SJF.txt fcfs_modif fcfs_modif.c round_robin_modif round_robin_modif.c sjf_modif sjf_modif.c
fahru@FAHRUR:~/praktikum4/tugas$ ./fcfs_modif

=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
ID    AT    BT    CT    TAT    WT
P1    0    24    24    24    0
P2    1     3    27    26    23
P3    2     6    33    31    25
P4    3     6    39    36    30

Rata-rata Waiting Time: 19.50
Rata-rata Turnaround Time: 29.25

Gantt Chart:
| P1 (0-24) | P2 (24-27) | P3 (27-33) | P4 (33-39) |

Hasil juga disimpan di file hasil_fcfs.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt Round_Robin.txt SJF.txt fcfs_modif fcfs_modif.c hasil_fcfs.txt round_robin_modif round_robin_modif.c sjf_modif sjf_modif.c
fahru@FAHRUR:~/praktikum4/tugas$ |
```

SJF

Masukkan data ke file input txt

```
fahru@FAHRUR: ~/praktikum
GNU nano 6.2
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Hasil


```
fahru@FAHRUR:~/praktikum4/tugas$ ./sjf_modif

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID    AT    BT    CT    TAT    WT
Gantt Chart:
| P1 (0-8) | P5 (8-11) | P2 (11-15) | P4 (15-20) | P3 (20-29) |

ID    AT    BT    CT    TAT    WT
P1     0     8     8     8     0
P2     2     4     15    13     9
P3     4     9     29    25    16
P4     6     5     20    14     9
P5     8     3     11     3     0

Rata-rata Waiting Time: 6.80
Rata-rata Turnaround Time: 12.60

Hasil juga disimpan di file hasil_sjf.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  Round_Robin.txt  SJF.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt  hasil_sjf.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c
```

ROUND ROBIN

Masukkan data ke file input txt

```
GNU nano 6.2
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
```

Atur time quantumnya menjadi 3

```
int n, quantum = 3;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
```

Hasil

```
fahru@FAHRUR:~/praktikum4/tugas$ ./round_robin_modif

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:
| P1 (0-3) | P2 (3-6) | P3 (6-9) | P4 (9-12) | P1 (12-15) | P3 (15-18) | P4 (18-21) | P1 (21-24) | P1 (24-27) | P1 (27-30) | P1 (30-33) | P1 (33-36) | P1 (36-39) |

ID    AT    BT    CT    TAT    WT
P1     0    24    39    39    15
P2     1     3     6     5     2
P3     2     6    18    16    10
P4     3     6    21    18    12

Rata-rata Waiting Time: 9.75
Rata-rata Turnaround Time: 19.50

Hasil juga disimpan di file hasil_rr.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  Round_Robin.txt  SJF.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt  hasil_rr.txt  hasil_sjf.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c
```

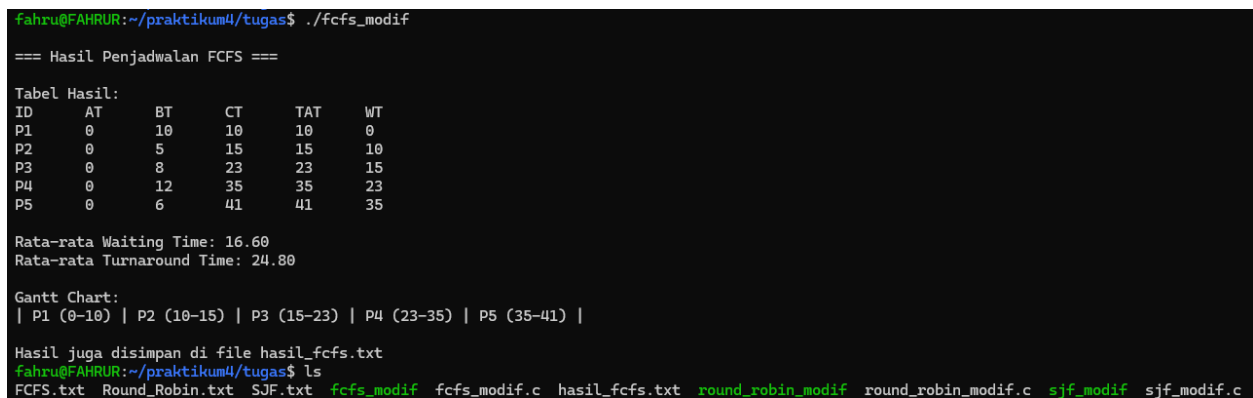
Test Case 2 Proses datang bersamaan

Ubah seluruh data masing-masing program seperti ini lalu jalankan programnya



```
fahru@FAHRUR: ~/praktikum
GNU nano 6.2
5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6
```

Hasil FCS



```
fahru@FAHRUR:~/praktikum4/tugas$ ./fcfs_modif

=== Hasil Penjadwalan FCFS ===

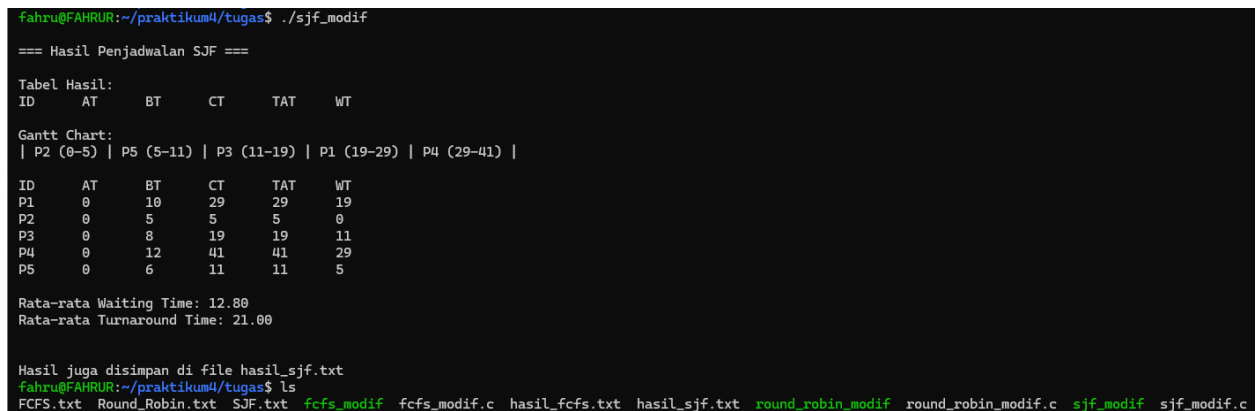
Tabel Hasil:
ID   AT   BT   CT   TAT  WT
P1   0   10   10   10   0
P2   0   5    15   15   10
P3   0   8    23   23   15
P4   0  12   35   35   23
P5   0   6    41   41   35

Rata-rata Waiting Time: 16.60
Rata-rata Turnaround Time: 24.80

Gantt Chart:
| P1 (0-10) | P2 (10-15) | P3 (15-23) | P4 (23-35) | P5 (35-41) |

Hasil juga disimpan di file hasil_fcfs.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt Round_Robin.txt SJF.txt fcfs_modif fcfs_modif.c hasil_fcfs.txt round_robin_modif round_robin_modif.c sjf_modif sjf_modif.c
```

Hasil SJF



```
fahru@FAHRUR:~/praktikum4/tugas$ ./sjf_modif

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID   AT   BT   CT   TAT  WT
P1   0   10   29   29   19
P2   0   5    5    5    0
P3   0   8   19   19   11
P4   0  12   41   41   29
P5   0   6   11   11    5

Rata-rata Waiting Time: 12.80
Rata-rata Turnaround Time: 21.00

Hasil juga disimpan di file hasil_sjf.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt Round_Robin.txt SJF.txt fcfs_modif fcfs_modif.c hasil_fcfs.txt hasil_sjf.txt round_robin_modif round_robin_modif.c sjf_modif sjf_modif.c
```

Atur Time Quantumnya menjadi 4

```

int n, quantum = 4;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}

```

Hasil ROUND ROBIN

```

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:
| P1 (0-4) | P2 (4-8) | P3 (8-12) | P4 (12-16) | P5 (16-20) | P1 (20-24) | P2 (24-25) | P3 (25-29) | P4 (29-33) | P5 (33-35) | P1 (35-37) | P4 (37-41) |

ID   AT   BT   CT   TAT  WT
P1   0    8   37   37   27
P2   0    5   25   25   20
P3   0    8   29   29   21
P4   0   12   41   41   29
P5   0    6   35   35   29

Rata-rata Waiting Time: 25.20
Rata-rata Turnaround Time: 33.40

Hasil juga disimpan di file hasil_rr.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt Round_Robin.txt SJF.txt  fcfs_modif  fcfs_modif.c  hasil_rr.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c
fahru@FAHRUR:~/praktikum4/tugas$

```

Test Case 3 Proses datang dengan interval

Masukkan data berikut ke dalam isi tiap masing-masing file input txt tiap program

5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3

Hasil FCFS

```

fahru@FAHRUR:~/praktikum4/tugas$ ./fcfs_modif

=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
ID   AT   BT   CT   TAT  WT
P1   0    8    8    8    0
P2   2    4   12   10    6
P3   4    9   21   17    8
P4   6    5   26   20   15
P5   8    3   29   21   18

Rata-rata Waiting Time: 9.40
Rata-rata Turnaround Time: 15.20

Gantt Chart:
| P1 (0-8) | P2 (8-12) | P3 (12-21) | P4 (21-26) | P5 (26-29) |

Hasil juga disimpan di file hasil_fcfs.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt Round_Robin.txt SJF.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c

```

Hasil SJF

```
fahru@FAHRUR:~/praktikum4/tugas$ ./sjf_modif

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID      AT      BT      CT      TAT      WT
Gantt Chart:
| P1 (0-8) | P5 (8-11) | P2 (11-15) | P4 (15-20) | P3 (20-29) |

ID      AT      BT      CT      TAT      WT
P1       0       8       8       8       0
P2       2       4       15      13       9
P3       4       9       29      25      16
P4       6       5       20      14       9
P5       8       3       11       3       0

Rata-rata Waiting Time: 6.80
Rata-rata Turnaround Time: 12.60

Hasil juga disimpan di file hasil_sjf.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  Round_Robin.txt  SJF.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt  hasil_sjf.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c
```

Atur time quantumnya menjadi 2

```
int n, quantum = 2;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
```

Hasil ROUND ROBIN

```
fahru@FAHRUR:~/praktikum4/tugas$ ./round_robin_modif

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:
| P1 (0-3) | P2 (3-6) | P3 (6-9) | P4 (9-12) | P5 (12-15) | P1 (15-18) | P2 (18-19) | P3 (19-22) | P4 (22-24) | P1 (24-26) | P3 (26-29) |

ID      AT      BT      CT      TAT      WT
P1       0       8      26      26      18
P2       2       4      19      17      13
P3       4       9      29      25      16
P4       6       5      24      18      13
P5       8       3      15       7       4

Rata-rata Waiting Time: 12.80
Rata-rata Turnaround Time: 18.60

Hasil juga disimpan di file hasil_rr.txt
fahru@FAHRUR:~/praktikum4/tugas$ ls
FCFS.txt  Round_Robin.txt  SJF.txt  fcfs_modif  fcfs_modif.c  hasil_fcfs.txt  hasil_rr.txt  hasil_sjf.txt  round_robin_modif  round_robin_modif.c  sjf_modif  sjf_modif.c
```

Tugas 5

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
Saat semua proses datang secara berurutan serta memiliki waktu eksekusi yang sama.
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
Karena SJF fokus terhadap proses yang paling singkat terlebih dahulu.

- Apa trade-off dari Round Robin?

Trade_off adalah suatu kondisi Dimana jika semakin kecil quantum maka sistem akan semakin responsif tapi akan banyak waktu yang terbuang untuk perghantian proses.jika semakin besar quantum sistem akan semakin efisien akan tetapi respon proses jadi lambat dan akan mirip seperti proses FCFS.

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
Algoritma yang paling banyak melakukan context switch adalag Round Robin
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
Pengaruhnya ialah semakin kecil quantumnya maka akan semakin banyak context switch yang terjadi,dan jika semakin besar quantumnya maka akan semakin sedikit context switchnya,akan tetapi sistem akan menjadi kurang responsif.
- Coba jalankan Round Robin dengan *quantum* 2, 4, 8, 16 - apa yang terjadi?

```

GNU nano 6.2
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3

```

▪ Quantum 2

```

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:
| P1 (0-3) | P2 (3-6) | P3 (6-9) | P4 (9-12) | P5 (12-15) | P1 (15-18) | P2 (18-19) | P3 (19-22) | P4 (22-24) | P1 (24-26) | P3 (26-29) |

ID   AT   BT   CT   TAT  WT
P1   0    8   26   26   18
P2   2    4   19   17   13
P3   4    9   29   25   16
P4   6    5   24   18   13
P5   8    3   15    7    4

Rata-rata Waiting Time: 12.80
Rata-rata Turnaround Time: 18.60

```

▪ Quantum 4

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:

| P1 (0-4) | P2 (4-8) | P3 (8-12) | P4 (12-16) | P5 (16-19) | P1 (19-23) | P3 (23-27) | P4 (27-28) | P3 (28-29) |

ID	AT	BT	CT	TAT	WT
P1	0	8	23	23	15
P2	2	4	8	6	2
P3	4	9	29	25	16
P4	6	5	28	22	17
P5	8	3	19	11	8

Rata-rata Waiting Time: 11.60

Rata-rata Turnaround Time: 17.40

▪ Quantum 8

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:

| P1 (0-8) | P2 (8-12) | P3 (12-20) | P4 (20-25) | P5 (25-28) | P3 (28-29) |

ID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	12	10	6
P3	4	9	29	25	16
P4	6	5	25	19	14
P5	8	3	28	20	17

Rata-rata Waiting Time: 10.60

Rata-rata Turnaround Time: 16.40

▪ Quantum 16

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:

| P1 (0-8) | P2 (8-12) | P3 (12-21) | P4 (21-26) | P5 (26-29) |

ID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	12	10	6
P3	4	9	21	17	8
P4	6	5	26	20	15
P5	8	3	29	21	18

Rata-rata Waiting Time: 9.40

Rata-rata Turnaround Time: 15.20

Kesimpulannya

Berdasarkan hasil percobaan dari gambar diatas, sistem menjadi lebih responsif jika quantum makin mengecil, tetapi akibatnya akan menjadi banyak context switch, lain halnya dengan sistem akan menjadi lebih efisien jika quantum makin besar, tetapi akan menjadi kurang interaktif.

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
Algoritma yang paling adil adalah round robin.
- Proses mana yang paling dirugikan di algoritma FCFS?
Proses yang paling dirugikan dalam algoritma FCS adalah proses yang datang belakangan dengan waktu eksekusi yang paling pendek.
- Apakah ada kemungkinan starvation di SJF?

Ada, biasanya terjadi pada proses besar yang datang lebih awal sebagai akibat dari proses kecil yang diprioritaskan.

4. Real World Application

- Untuk *web server* yang melayani banyak kecil, algoritma mana yang cocok?

Round Robin cocok untuk web server yang melayani bannyak permintaan kecil respon yang diberikan darinya cepat dan adil pada setiap permintaan, sehingga semua pengguna terlayani hampir bersamaan.

- Untuk *render video* (task besar), algoritma mana yang cocok?

Algoritma yang cocok untuk render video adalah FCFS karena mampu menjalankan proses besar dengan tanpa gangguan secara penuh serta meminimalisir overhead dari pergantian proses.

- Untuk OS *desktop* (interaktif), algoritma mana yang cocok?

Algoritma yang cocok untuk OS desktop adalah round robin karena dapat memberikan respon yang cepat ,adil, serta nyaman bagi pengguna.